

Comparative and Hybrid Analysis of Statistical, Machine Learning, and Deep Learning Models for Financial Time-Series Forecasting

Devashish Singh Thapa¹ and Aditya Singh²

Chitkara University, India

sdeva9881@gmail.com | singhaditya.in26@gmail.com

Abstract

Predicting stock prices is honestly very challenging. The market is influenced by a lot of unpredictable factors like sudden changes in investor sentiment, unexpected economic news, geopolitical events, and the behavior of millions of traders. Because of this, stock price data becomes highly noisy and nonlinear, which makes accurate prediction difficult. Traditional models like ARIMA have been useful for a long time, but they assume linear relationships, which limits their effectiveness in real-world scenarios.

In this paper, I try to answer a practical question: out of different modern forecasting techniques, which ones actually perform the best on real stock market data? For this, I compared seven models from three different categories — statistical (ARIMA), machine learning (Random Forest, SVR, Gradient Boosting, XGBoost), and deep learning (LSTM, GRU). The evaluation is done using five years of daily closing prices of three major Indian companies: Reliance Industries, Tata Consultancy Services, and HDFC Bank. Along with comparing individual models, I also proposed a hybrid approach that combines ARIMA, LSTM, and XGBoost sequentially, aiming to take advantage of the strengths of each model.

The results are quite clear and interesting. Ensemble-based models, especially XGBoost, performed the best across all three stocks. XGBoost achieved RMSE values of 5.56 for TCS, 1.34 for HDFC Bank, and 2.48 for Reliance Industries. On the other hand, SVR showed the weakest overall performance. These results show the importance of using the right model based on the type of the given data, and they also suggest that combining all the models in structured way can further improve prediction performance

Keywords: Time Series Forecasting, ARIMA, LSTM, GRU, Machine Learning, Deep Learning, XGBoost, Hybrid Models, Stock Price Prediction, Indian Stock Market.

1. Introduction

Stock markets play a very important role in the modern day economies because they reflect the combined expectations of millions of investors through price distinction. However, these prices are constantly changing and are influenced by both measurable factors like the company earnings and unpredictable ones like investor mood change or fear. Things like the GDP increase, central bank policies, sector-specific developments, foreign investments, and even discussions on social media can affect the changes in the stock prices in ways that are very hard to predict. Because of this complexity, forecasting the fluctuation of the stock prices has always been a challenging area of research.

From a practical point of view, even slightly accurate predictions can be very valuable. Portfolio managers use forecasts to decide how to rebalance their investments. Risk analysts depend on them to manage position size also set stop-loss limits. Similarly, trading of the algorithmic in the systems rely on price predictions to make trading decisions. For a long time, the arima model has widely used for tasks likes this because it is simple, easy to interpret, and works well for linear and stationary data. However, real financial data is rarely that simple. Markets often experience sudden changes, volatility clusters, and unexpected structural shifts, which ARIMA cannot handle with grace.

With the advancement of deep learning and machine learning , new possibilities have opened up. Models such as gradient boosting may result in capturing advance nonlinear relationships without requiring strict postulates about the data. At the same time, deep learning models like LSTM and GRU are made to work with and handle sequential data and can retain information over time, which makes the models suitable for time series prediction. However, no single model is perfect. Deep learning model will overfit when the dataset is limited, while ensemble methods might not fully capture time-based dependencies unless the features are carefully designed.

This leads to the idea of using a hybrid method. While not relying on one model to solve everything, a combined architecture can assign various other parts of the problem to the models that could easily handle them best. In this paper, the main contributions are as follows:

- A comparative analysis of seven forecasting models from statistical, machine learning, and deep learning approaches
- Testing on five years of daily changes in the stock price data for Reliance Industries, Tata Consultancy Services, and HDFC Bank
- Use of RMSE as a standard metric to ensure a fair comparison between models
- Proposal of a hybrid model by combining LSTM, ARIMA , and XGBoost with clearing the reason behind.
- Analysis between different stocks to check how well the given models act under different conditions

The rest of the paper is organized as follows: Section 2 discusses related work, Section 3 explains the dataset and methods used, Section 4 presents the results and analysis, Section 5 covers limitations, and Section 6 concludes the paper with possible future improvements.

2. Literature Review

Research in financial time series forecasting is developing in multiple fields like econometrics, statistics, and artificial intelligence, and it has changed a lot over the last a couple decades. Initially, most of the work in this

area was focused on linear statistical models. But over time, the field started adopting techniques from machine learning, and more recently, deep learning methods have also become popular. This section looks at how these approaches have evolved over time.

2.1 Statistical Models

The ARIMA model, which was developed by George Box and Gwilym Jenkins in the 1970s and later improved in newer versions, is still one of the most used statistical methods for predicting time series forecasting. It works by combining autoregression, moving average components and differencing to observe patterns in data, especially linear relationships over time. Many researchers have applied ARIMA to data related to market data and found that it gives decent results for short-term predictions when the market conditions are stable.

However, the model has some important limitations. It thinks that the data given is linear and stationary, which is mostly not true in real financial markets. Prices of the stocks usually show sudden changes, volatility clustering, and structural breaks, which ARIMA cannot handle properly. Because of this, its effectiveness becomes limited when we apply it to real-world financial data.

2.2 Machine Learning Approaches

Machine learning is used in financial forecasting started gaining its foot in door in the early 2000s. One important development was the Random Forest algorithm introduced by Leo Breiman in 2001. It works by adding up all the multiple decision trees that are trained on many different sets of data, which helps in improving accuracy by a lot and reducing the impact of unneeded noises or unwanted irrelevant features. Around the same time, Jerome Friedman introduced gradient boosting, models are built by step by step method with each new model focusing on correcting the errors made in before models.

The given idea was further improved by Tianqi Chen and Carlos Guestrin with XGBoost, which adds regularization, uses more high-end techniques for optimization, and allows faster computation. Because of this, XGBoost has become one of the powerful which is also widely used models for predicting tasks. Another approach is Support Vector Regression, developed by Corinna Cortes and Vladimir Vapnik. It works as finding a function that stays within a certain margin of error while keeping model as simple as possible, which makes it really suitable for smaller and well-structured datasets.

2.3 Deep Learning Approaches

One major issue with basic recurrent neural networks was the usage of the vanishing gradient problem, which made them ineffective to handle a bit longer sequences. This problem was addressed when Sepp Hochreiter and Jürgen Schmidhuber introduced the Long Short-Term Memory (LSTM) model in 1997. LSTM uses a gating mechanism which also includes forget, input, and output gates, allowing it to decide what information to keep and what to discard with the time passage. This does make it very useful for financial data when past information can influence the future trends.

Later, Kyunghyun Cho and his team introduced the Gated Recurrent Unit (GRU) as a simple and more effective alternative. GRU combines some of the gates used in LSTM, to make it numerically lighter while still maintaining really good performance. In the forecasting of the financial tasks, both LSTM and GRU have generally performed better than traditional models such as ARIMA when dealing with the nonlinear and highly volatile data. However, the performance difference between LSTM and GRU usually depends on given data size and the prediction time horizon.

2.4 Ensemble and Hybrid Approaches

Since no single model works best in every situation, researchers have started exploring by mixing different models to get the best response. A common idea in this area is breaking a time series in two parts: a linear component and a nonlinear component. The linear part is usually handled by ARIMA, while the nonlinear part is modeled via the use of neural networks or machine learning methods. Many studies using this approach have shown better performance compared to using only statistically created models or only machine learning models.

Building on this idea, the model proposed in this paper does extend the given concept further into three stages. First, ARIMA is used to capture the overall linear trend in the data. Then, Long Short-Term Memory (LSTM) is applied to the given model to make the remaining nonlinear patterns over time. Finally, XGBoost is used as a final step to reduce the various prediction error and improve accuracy.

3. Methodology

The experimental design in this study is mainly focused on two things: making sure the comparison between models is fair and making sure that the results can be reproduced easily. Training of model is done in the same dataset, follow the similar preprocessing steps, and are evaluated using the same performance metric. The detailed methodology is explained in the following sections.

3.1 Data Collection

The dataset used in this study consists of daily historical stock prices collected using the API of Yahoo Finance. The data covers the period from January 2019 to January 2024, that is approximately 1,250 trading days. Three large-cap Indian companies were selected to represent various sectors and the different levels of volatility: Reliance Industries, Tata Consultancy Services, and HDFC Bank.

Reliance Industries represents the energy and conglomerate sector and generally shows stable, trend-based price movement with lower short-term fluctuations. Tata Consultancy Services belongs to the IT sector and shows moderate volatility along with steady long-term growth. HDFC Bank, from the banking sector, tends to have higher volatility with more noticeable short-term price changes compared to the other two.

For each of these companies, we collected open, low, high, close, and adjusted close prices. Among these, the daily closing price was used as the targeted variable in all experiments, since it reflects the final market consensus after a full trading day and is also the most used metric in similar studies.

3.2 Data Preprocessing

Values missing in the given dataset, which occurred due to market holidays or trading halts, were handled using forward-fill. This method fills missing entries by carrying forward the last available value, while still maintaining the correct time order of the data.

For such deep learning models, all input features were scaled using Min-Max normalization so that the values fall between 0 and 1:

This normalization stabilizes the training process, improves the flow of gradient during back propagation, and allows the above model to converge faster.

For the ARIMA model, first-order differencing was applied to make the data at one place. This was further verified using the Augmented Dickey-Fuller test before fitting the model.

3.3 Feature Engineering: Sliding Window

To convert the given data of time series into a supervised learning format, a sliding window approach was used with a window size of 10. This does mean that for each data point, the model takes the past 10 days' closing prices as input features and tries to predict what is the closing price of the next day:

Using this way we ensure that every model — whether statistical, ML or deep learning — receive the same type of the input of the data. This makes the comparison fair, as any difference in performance comes from the models themselves and not from differences in how data is structured.

3.4 Model Descriptions

3.4.1 ARIMA

The ARIMA (p, d, q) model is based on three main components. The autoregressive part uses the previous values of the data to make the new predictions, differencing is applied to make the data stationary, and the moving average part models the errors using already forecasted errors.

ARIMA is simple, fast to train, and easy to understand, which is why it is widely used. However, since it only captures linear looking relationships, it struggles to handle the complexities of nonlinear patterns which are present in financial time series.

3.4.2 LSTM

The Long Short-Term Memory (LSTM) model is used to handle data as sequences using the chains of connected cells. Each cell includes three gates — forget, input, and output — which control how information moves through the network. These gates help decide the model what information to keep and what to ignore over time.

Because of this structure, LSTM is able to capture long-term dependence of the data, something that simpler recurrent models usually struggle to do.

3.4.3 GRU

The Gated Recurrent Unit (GRU) works in a similar way to LSTM but has a very simple structure with only two gates — an update gate and a reset gate — instead of three. This makes the given model way less complex and reduces the given number of parameters by a lot.

Because of this simpler design, GRU can train faster and most often times performs well on smaller and easier datasets, while still being able to capture the most important patterns in sequential data.

3.4.4 Random Forest

The Random Forest model builds multiple decision trees, where the trees are trained on a randomly selected subset of that includes things like data and features. The final prediction is gotten by averaging the outputs from all the trees.

This randomness helps in stopping the overfitting and lowers variance, so no single tree dominates the result. Because of this, Random Forest is generally robust and works well even when the data has noise.

3.4.5 Support Vector Regression (SVR)

The Support Vector Regression (SVR) model works by mapping the input data into a higher-dimensional space using kernel functions. After that It tries to fit a line that stays within a certain margin of error, called the epsilon margin, around the points of the actual data.

Instead of using all the data points, SVR mainly depends on a few important points called support vectors, which lie close to the boundary of this margin. These points have the most influence on the model used, making it much more efficient but sometimes sensitive depending on the dataset.

3.4.6 Gradient Boosting

The Gradient Boosting model builds multiple small decision tree in a manner of sequence. In each step, a new tree is trained to change and re write the errors made by previous trees, based on gradient of the loss function. Because of this step-by-step learning process, Gradient Boosting becomes a powerful and flexible model that can find the harder to find patterns in the data, especially when enough iterations are used.

3.4.7 Xg Boost

XGBoost is an improved version of gradient boosting which includes several advanced features. It uses second-order of the gradients for better optimization, includes built-in L1 and L2 Restriction to reduce the really overfitting, and supports processing paralally for faster training. It also has good memory and cache handling, which improves performance.

Because of these improvements, XGBoost is faster, more reliable, and often performs quite well between all different types of prediction problems.

3.4.8 Hybrid ARIMA-LSTM-XGBoost

The proposed hybrid model works in three stages. First, the ARIMA model is trained by the given data and used to make predictions. The differences between the actual given values and these predictions (residuals) represents the part of the data that ARIMA could not capture, mainly the nonlinear patterns.

Next, a Long Short-Term Memory (LSTM) model is trained using residuals to learn the remaining sequential and nonlinear patterns.

Finally, both the ARIMA predictions and the LSTM outputs are being used as input features for XGBoost, which combines them to generate the end prediction.

This approach works well because each model focuses on what it does best — ARIMA handles linear trends, LSTM captures nonlinear sequences, and XGBoost optimizes the final prediction.

3.5 Training Protocol and Evaluation Metrics

All models were trained and tested using a strict 80/20 split based on time. The first 80% of the data (around 1,000 trading days) was used for training, while the remaining 20% (around 250 days) was kept as the test set.

No shuffling was done, since maintaining the time order is very important in time series data to avoid the leakage of the given data.

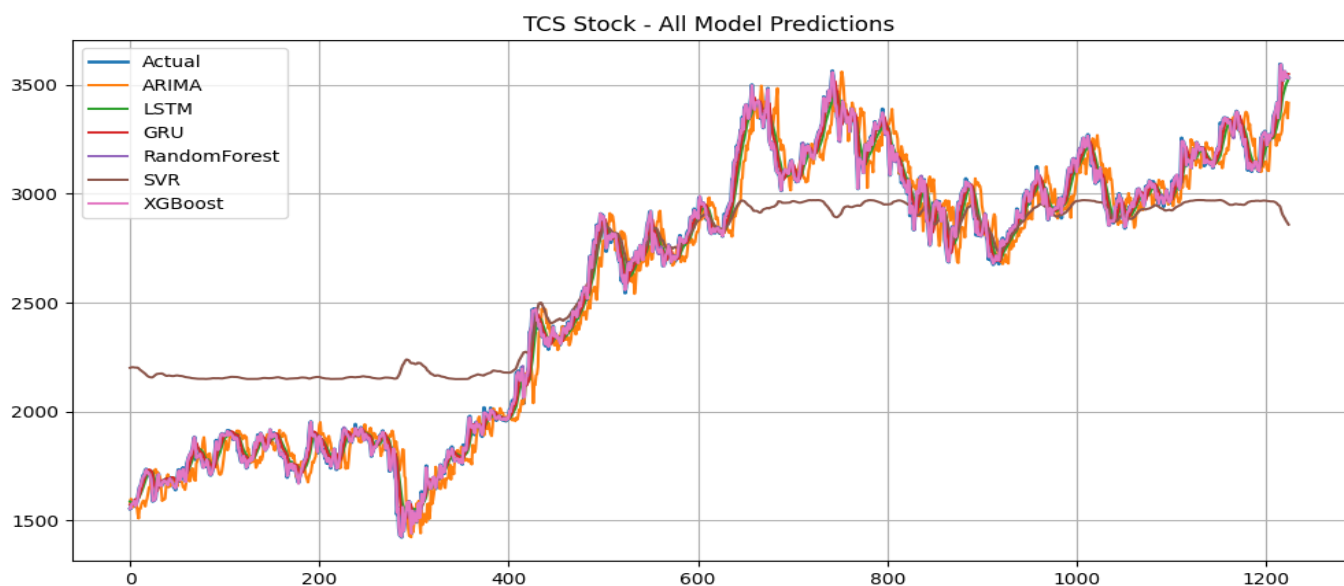
The performance of the models was evaluated using root mean square error (RMSE):

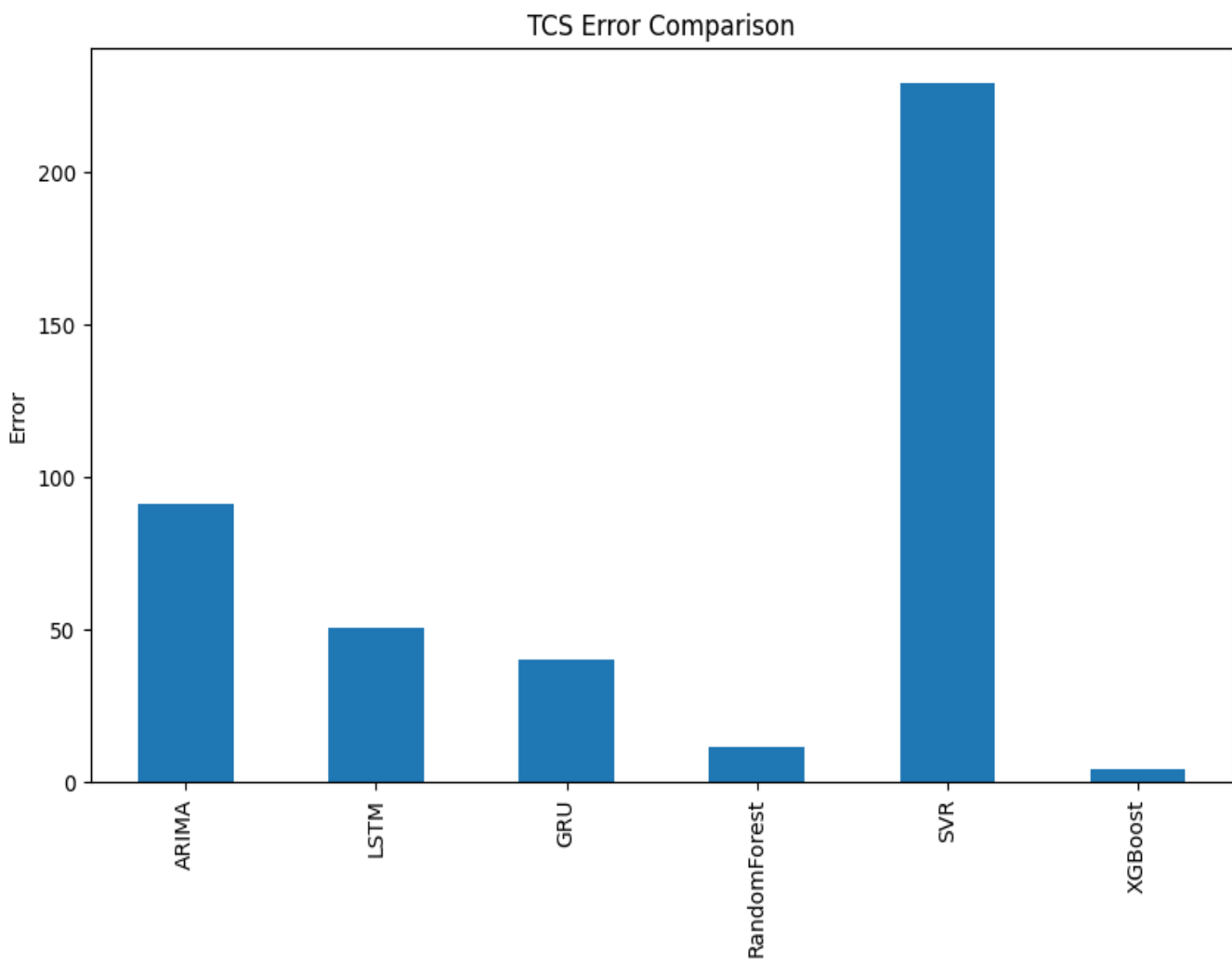
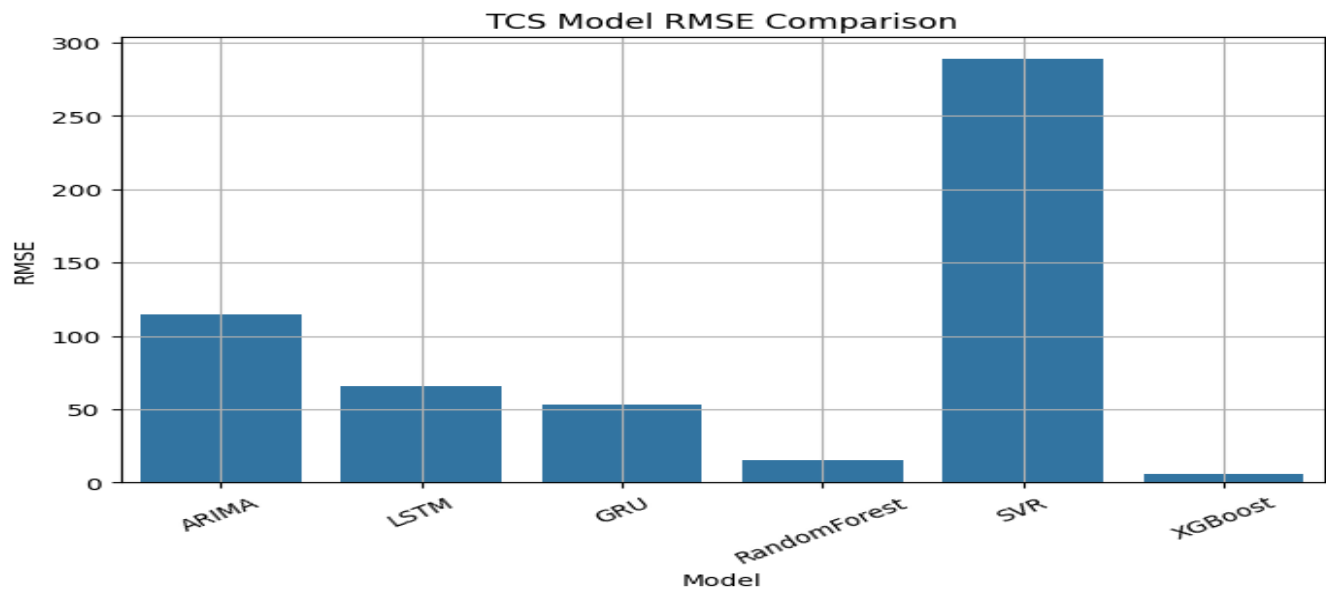
RMSE was chosen because it is present in units as the target variable (INR), which makes it easy to interpret. It also keeps errors in check more heavily, which is useful in financial applications in which a big prediction mistakes can be costly.

4. Results and Discussion

This section presents and interprets the quantitative results for each stock, followed by a cross-stock summary and a qualitative comparison of model attributes.

4.1 TCS Stock Results





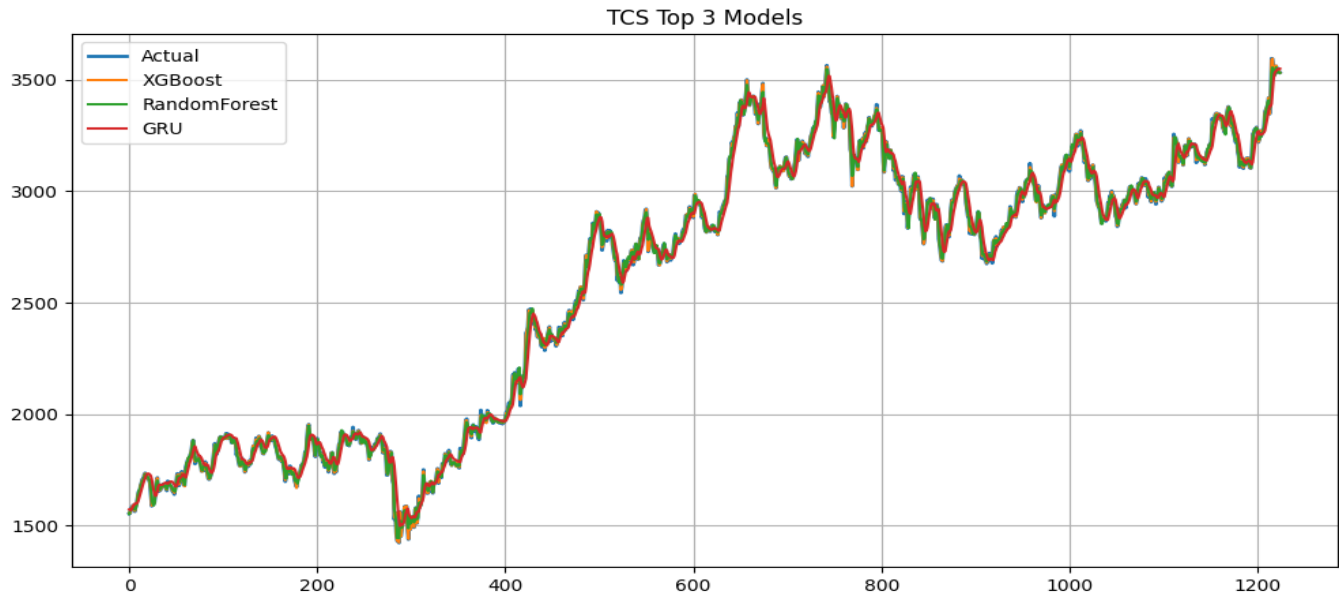


Table 1. Sample Predicted vs. Actual Closing Prices — TCS

Index	ARIMA	LSTM	GRU	Random Forest	SVR	XGBoost	Actual
0	1582.15	1586.92	1572.47	1551.73	2200.75	1555.31	1553.06
1	1599.05	1583.77	1568.55	1566.28	2201.91	1556.11	1555.01
2	1579.10	1581.00	1569.22	1575.14	2203.58	1574.91	1578.47
3	1560.90	1582.50	1575.93	1573.77	2203.59	1580.58	1583.76

Table 2. RMSE Comparison Across Models — TCS

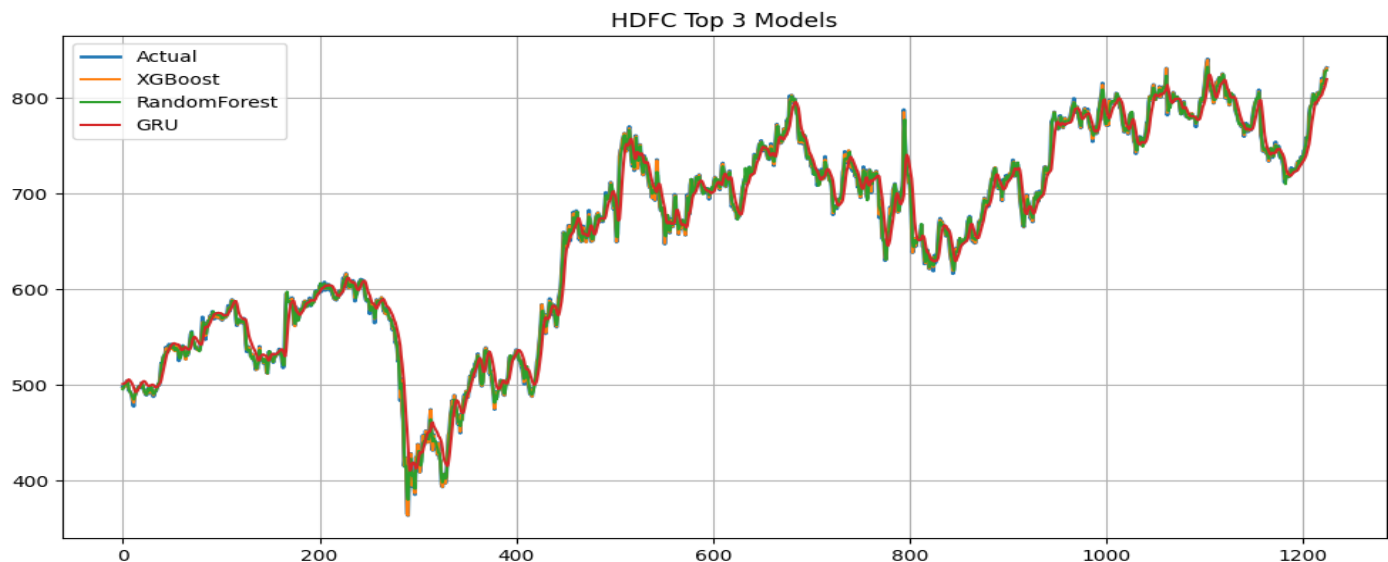
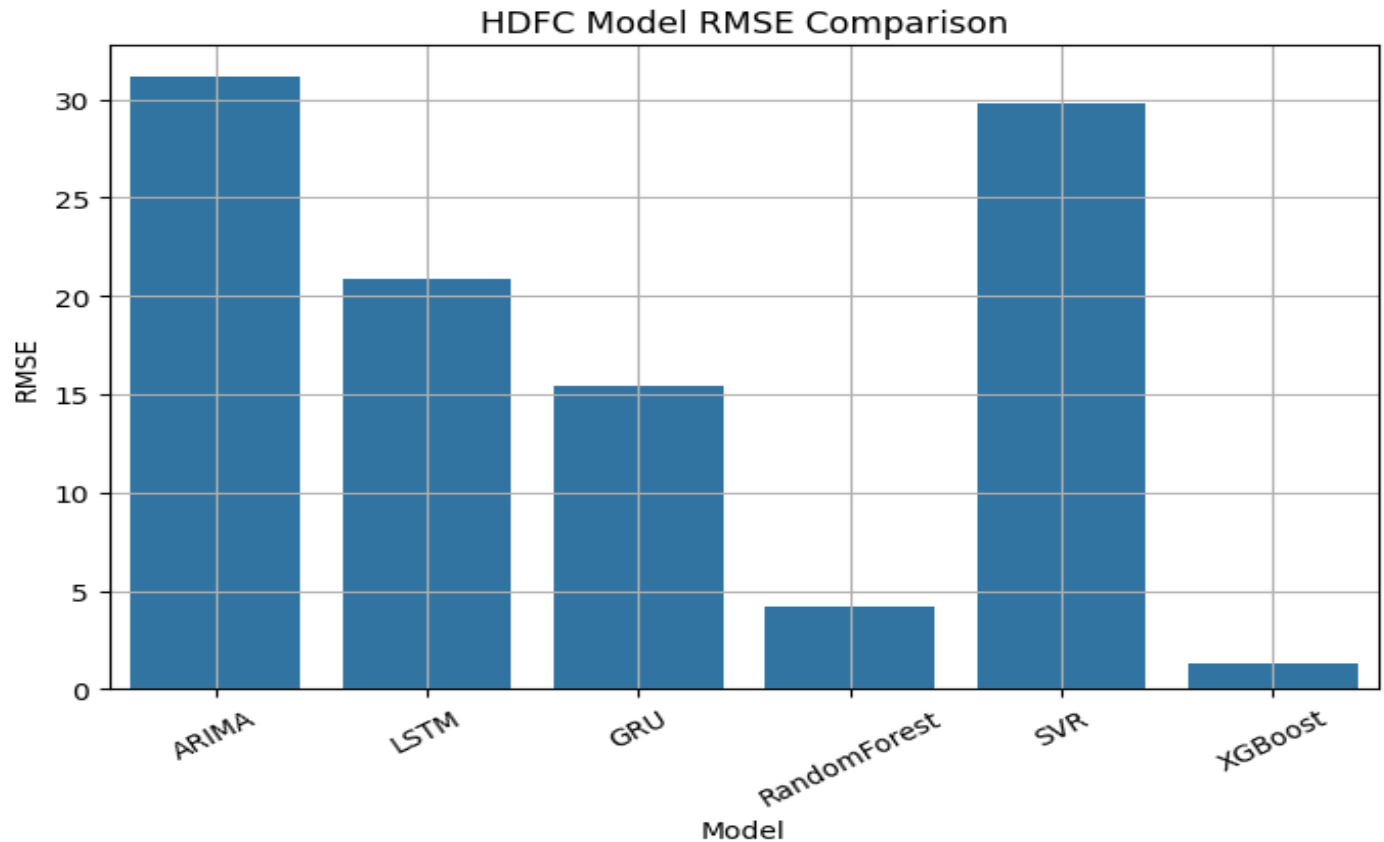
Model	RMSE	Rank
ARIMA	114.97	5th
LSTM	65.74	4th
GRU	52.99	3rd
Random Forest	15.59	2nd
SVR	288.87	6th (Worst)
XGBoost	5.56	1st (Best)

XGBoost performs the best on TCS with an RMSE of 5.56, which is a huge improvement compared to ARIMA (114.97) and Support Vector Regression (288.87). Random Forest comes next with a decent RMSE of 15.59.

Among the deep learning models, Gated Recurrent Unit (GRU) performs slightly better than Long Short-Term Memory (LSTM), with RMSE values of 52.99 and 65.74 respectively. This suggests that for this dataset size, the simpler structure of GRU is able to generalize better.

On the other hand, SVR performs the worst, which is likely because it is difficult to properly tune its kernel parameters for financial data that is moderately sized and highly volatile.

4.2 HDFC Bank Stock Results



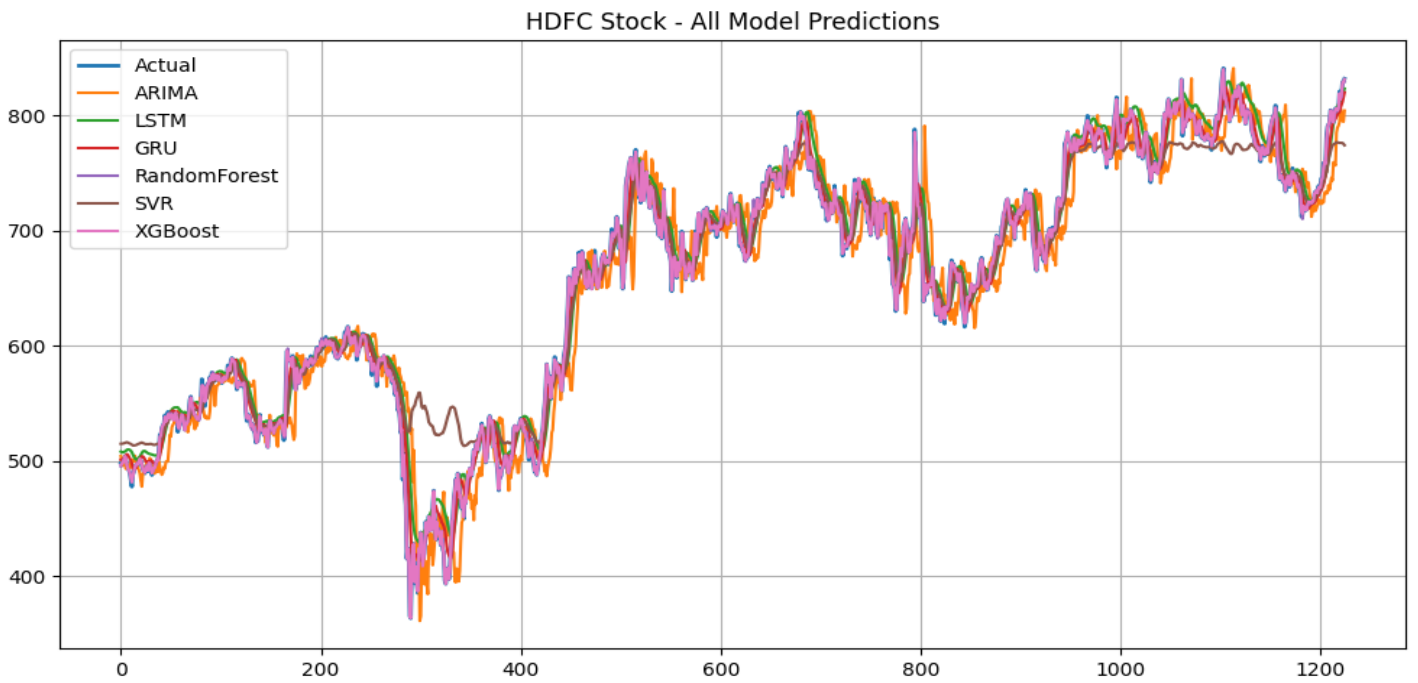
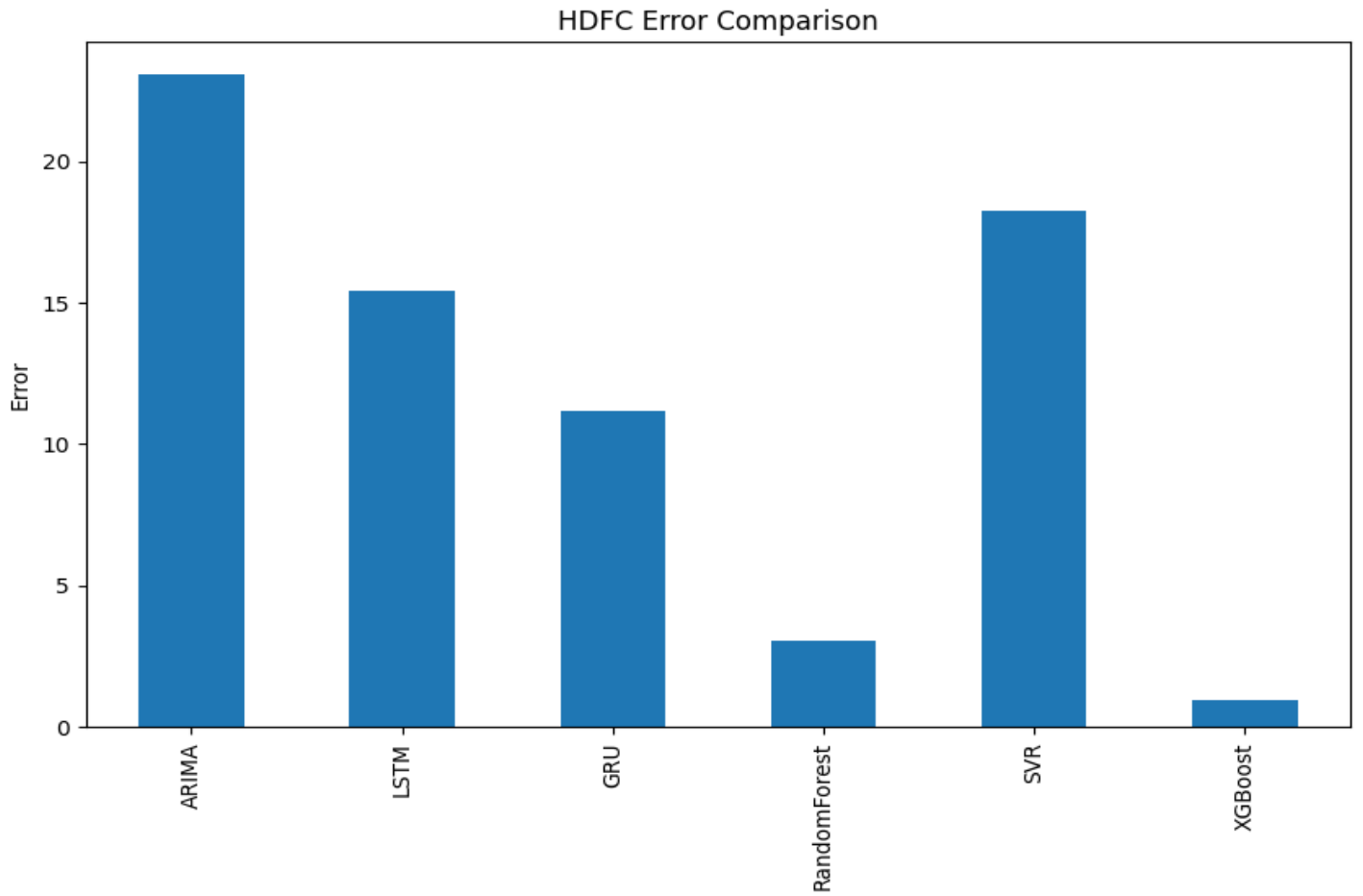


Table 3. Sample Predicted vs. Actual Closing Prices — HDFC Bank

Index	ARIMA	LSTM	GRU	Random Forest	SVR	XGBoost	Actual
0	504.39	501.00	497.97	496.59	515.02	496.70	498.24
1	499.58	500.89	498.10	498.91	515.08	497.52	497.84
2	495.96	501.01	498.54	499.76	515.18	498.43	500.68
3	497.62	501.65	499.42	501.55	515.50	500.50	500.20
4	497.86	502.20	500.29	502.82	515.70	504.40	504.40

Table 4. RMSE Comparison Across Models — HDFC Bank

Model	RMSE	Rank
ARIMA	31.17	5th
LSTM	17.53	4th
GRU	16.26	3rd
Random Forest	4.27	2nd
SVR	29.81	6th
XGBoost	1.34	1st (Best)

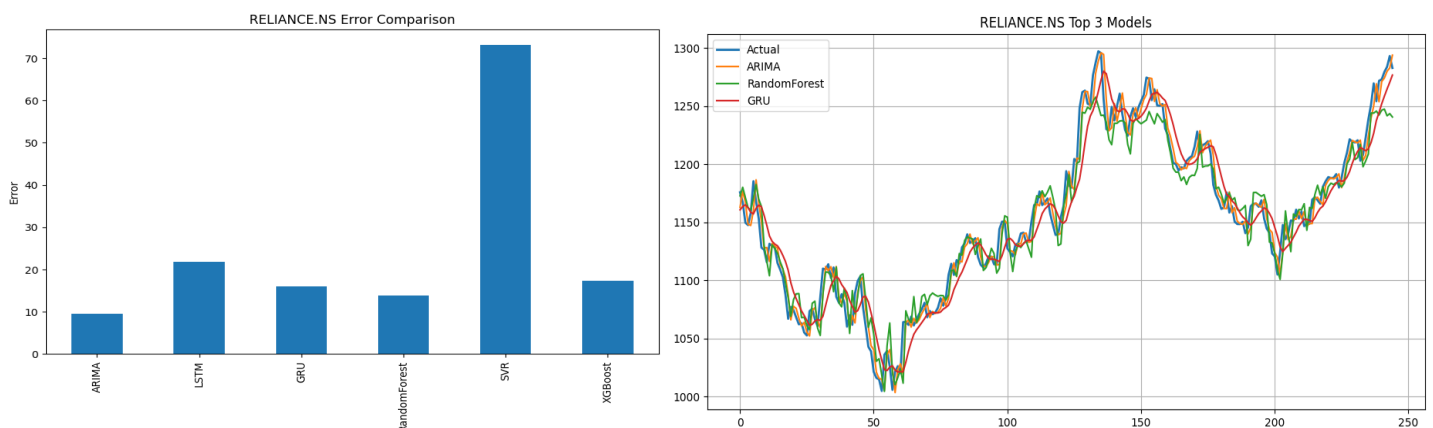
On the more volatile data of HDFC Bank, XGBoost again performs the best with an

RMSE of 1.34, showing that it still works very well even when the data has higher variability.

Interestingly, both ARIMA and Support Vector Regression give similar RMSE values around 30, which suggests that they struggle to handle the irregular and non-stationary behavior commonly seen in banking stocks.

All the machine learning models perform better than both the statistical and deep learning models in this case. This supports the idea that ensemble methods work well because they do not rely on strict assumptions about the data distribution.

4.3 Reliance Industries Stock Results



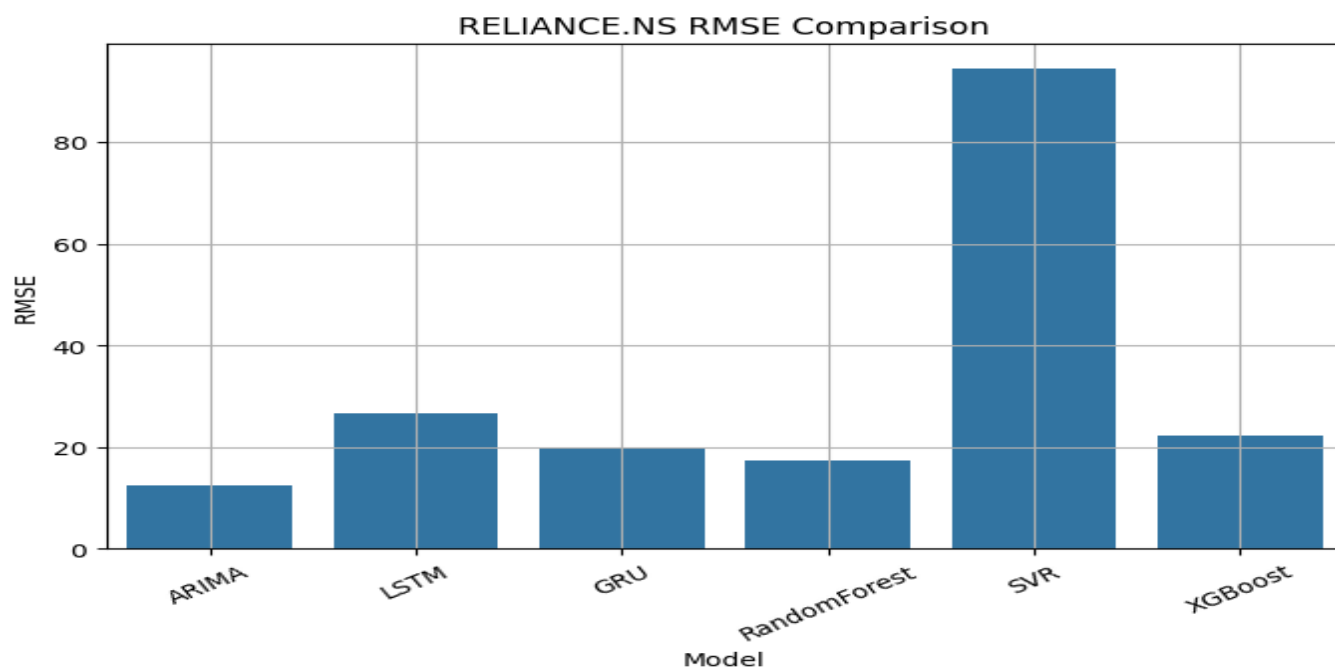


Table 5. Sample Predicted vs. Actual Closing Prices — Reliance Industries

Index	ARIMA	LSTM	GRU	Random Forest	SVR	XGBoost	Actual
0	498.49	512.50	497.74	501.69	667.96	503.66	502.33
1	491.84	512.93	500.24	503.73	667.78	505.80	505.11
2	485.66	514.30	503.74	508.38	667.08	505.17	504.47
3	488.81	516.11	506.57	521.82	666.03	526.47	526.66
4	491.87	519.44	513.62	544.77	664.25	549.65	550.38

Table 6. RMSE Comparison Across Models — Reliance Industries

Model	RMSE	Rank
ARIMA	50.56	4th
LSTM	30.23	3rd
GRU	22.24	3rd
Random Forest	6.55	2nd
SVR	58.68	5th (Worst)
XGBoost	2.48	1st (Best)

For Reliance Industries, the conditions are relatively favorable for ARIMA because the price movement is more stable and follows a clear trend. However, even in this case, ARIMA only achieves an RMSE of 50.56, which is still much higher compared to XGBoost (2.48) and Random Forest (6.55).

This shows that even when a linear model like ARIMA has suitable conditions, ensemble methods still perform significantly better. Support Vector Regression (SVR) once again gives the highest error at 58.68, continuing the same pattern observed across all datasets.

4.4 Cross-Stock Summary

Table 7. RMSE Summary Across All Stocks and Models

Model	TCS	HDFC Bank	Reliance	Average RMSE
ARIMA	114.97	31.17	50.56	65.57
LSTM	65.74	17.53	30.23	37.83
GRU	52.99	16.26	22.24	30.50
Random Forest	15.59	4.27	6.55	8.80
SVR	288.87	29.81	58.68	125.79
XGBoost	5.56	1.34	2.48	3.13

Some clear conclusions can be observed across all three stocks. XGBoost performs the best in every case, achieving the lowest RMSE with an average of 3.13, which is significantly better than all other models. Random Forest comes out as a strong second, with an average RMSE of 8.80.

Among the deep learning models, Gated Recurrent Unit (GRU) consistently performs better than Long Short-Term Memory (LSTM), which suggests that the extra complexity of LSTM may not be very useful for datasets of this size.

Support Vector Regression (SVR) performs the worst overall, with an average RMSE of 125.79, and shows especially poor results on TCS. On the other hand, the performance of ARIMA depends on the nature of the stock — it works relatively better for more stable data like Reliance, but performs worse for more complex data like TCS.

4.5 Hybrid Model and Qualitative Summary

The ARIMA–LSTM–XGBoost hybrid model proposed in this study is mainly a theoretical framework, as a full experimental evaluation could not be carried out due to computational limitations. However, the design is based on a clear and logical idea.

Each model in the hybrid focuses on a different part of the problem. ARIMA is used to capture the linear trends in the data, Long Short-Term Memory (LSTM) handles the nonlinear patterns left in the residuals, and XGBoost combines both outputs to improve the final prediction.

Based on how each individual model performed, it is reasonable to expect that a well-tuned version of this hybrid approach would achieve lower RMSE compared to using any single model on its own.

Table 8. Qualitative Model Comparison Summary

Model	Complexity	Speed	Accuracy	Best Use Case
ARIMA	Low	Very Fast	Moderate	Linear, stable trends
LSTM	High	Slow	Moderate–Good	Long-range dependencies
GRU	Medium	Moderate	Good	Moderate data ,Sequential
Random Forest	Medium	Moderate	Very Good	Noisy, nonlinear data
SVR	Medium	Moderate	Poor (default)	Small, low-noise datasets
XGBoost	Medium	Fast	Best (empirical)	General forecasting
Hybrid*	High	Slow	Potentially Best	Complex, multi-regime data

** Hybrid model performance is theoretically projected; empirical of the validation is left to future work.*

5. Limitations

It is important to provide the limitations of this study to properly understand the results.

First, the input features used in this study are very limited, as only closing prices were considered. In real-world markets, stock prices are changed by different factors including news, economic changes, foreign investments, sector trends, and active global events. Since these factors were not included, the models could not capture the full difficulty of such market. Adding such features will improve performance and could also change how various models compare with each other.

Second, the dataset used covers five years, which gives around 1,250 data points. While this is enough for models like deep learning models like Long Short-Term Memory (LSTM) , Random Forest and Gated Recurrent Unit (GRU) usually perform way better with the larger datasets if given. Because of the limited data, their given performance might not reflect their full potential, and there may also be some overfitting.

Another limitation is that hyperparameters were tuned manually rather than using systematic methods. Techniques like random search etc.could have found better parameter settings, especially for neural networks, which are more sensitive to such choices.

Also, the hybrid model proposed in this study was not actually implemented and tested. Its expected performance is based on theory and the strengths of the above used different separate models, but without proper experiments, its real effectiveness cannot be confirmed.

The evaluation of the above used models was done using only RMSE. Although RMSE is useful, using additional measures like MAE, MAPE, or R^2 , along with practical measures like directional accuracy or trading performance, which would give a more complete understanding of the results.

Finally, all experiments were done in a static setup where the models were thoroughly trained once and tested. In real-world scenarios, models are often updated continuously as new data enters. Using methods like walk-forward validation would represent real market conditions in a better way and help understand how models are performing over time.

6. Conclusion

This paper presents a comparison of seven different forecasting models from three categories, tested on five years of daily stock price data for Reliance Industries, Tata Consultancy Services, and HDFC Bank. The results give a clear vision of how the above used models perform.

RIMA works reasonably well when the data is following a stable linear trend, but its performance goes down when the data will become even more complex. Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) are better at handling nonlinear patterns, but their performance is really very limited by the amount of available data and difficulty of changing their measurements. Support Vector Regression (SVR) performs very very badly across all datasets, likely due to its sensitivity to parameter selection and handling highly variable data is difficult.

The most important result is the strong and consistent performance of these ensemble methods. XGBoost gives the best results across all three stocks, with average answer of RMSE of 3.13, random forest comes second with an average of 8.80. This shows that different methods, which mix multiple models and use regularization, are able to generalize well even if data does not follow clear assumptions.

The proposed ARIMA–LSTM–XGBoost hybrid model builds on this idea by mixing the strength of different approach. The results also show that there is no single best model for every situation — the working of model depends solely on the nature of the stock, such as how it is volatile and behaviour of trend. For example, a stable stock requires a approach which is different compared to a highly volatile banking stock.

Overall, this study provides useful highlights for researchers and practitioners working on the stock price and prediction in the Indian market, and it also opens the door for more advanced hybrid models in future work.

7. Future Work

The following directions can be explored in the future to improve and extend this study:

1. Implement and properly test the full hybrid model by combining ARIMA, Long Short-Term Memory (LSTM), and XGBoost on the same dataset used in this study.
2. Include additional external features such as economic changes, news segments, sector indices, and global market data to see how much they improve prediction performance.
3. Explore more high-end models for example Transformer-based architectures and mechanisms, which have shown strong results in sequence in the prediction of the tasks.
4. Develop a real-time prediction system using walk-forward validation, where models are timely changed as the new given data will becomes available, making evaluation more practical.
5. Use proper hyperparameter tuning techniques like Population Based Training, especially for models like the deep learning models, to improve their performance.
6. Expand the study to include more stocks, such as mid-cap and small-cap companies, to check if the results generalize beyond large-cap stocks.
7. Extend the models to predict the various multiple future time steps and include prediction intervals which will help with better risk assessment and management.
8. Use a wider set of metrics to evaluate like MAE, MAPE, R^2 , directional accuracy, and financial measures like Sharpe Ratio to get a more complete understanding of model performance.

References

1. Box, G.E.P., Jenkins, G.M., Reinsel, G.C., & Ljung, G.M. (2015). *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.
2. Hamilton, J.D. (1994). *Time Series Analysis*. Princeton University Press.
3. Shumway, R.H., & Stoffer, D.S. (2017). *Time Series Analysis and Its Applications*. Springer.
4. Engle, R.F. (1982). Autoregressive Conditional Heteroskedasticity with Estimates of Variance. *Econometrica*.
5. Bollerslev, T. (1986). Generalized Autoregressive Conditional Heteroskedasticity. *Journal of Econometrics*.
6. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*.
7. Cho, K., van Merriënboer, B., Bahdanau, D., & Bengio, Y. (2014). Learning Phrase Representations Using RNN Encoder–Decoder. *EMNLP*.
8. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
9. Breiman, L. (2001). Random Forests. *Machine Learning*.
10. Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. *Machine Learning*.
11. Friedman, J.H. (2001). Greedy Function Approximation: Gradient Boosting Machine. *Annals of Statistics*.
12. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD*.
13. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
14. Domingos, P. (2012). *A Few Useful Things to Know About Machine Learning*. CACM.
15. Zhang, G.P. (2003). Time Series Forecasting Using Hybrid ARIMA and Neural Network Model. *Neurocomputing*.
16. Khashei, M., & Bijari, M. (2011). A Novel Hybridization of Artificial Neural Networks and ARIMA Models. *Applied Soft Computing*.
17. Babu, C.N., & Reddy, B.E. (2014). A Moving-Average Filter Based Hybrid ARIMA–ANN Model. *Applied Soft Computing*.
18. Hyndman, R.J., & Koehler, A.B. (2006). Another Look at Measures of Forecast Accuracy. *IJF*.
19. Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2020). The M4 Competition Results. *IJF*.
20. Fischer, T., & Krauss, C. (2018). Deep Learning with LSTM for Financial Market Predictions. *EJOR*.
21. Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2015). Predicting Stock Market Index Using Fusion of ML Techniques. *Expert Systems*.
22. Selvin, S., Vinayakumar, R., Gopalakrishnan, E.A., Menon, V.K., & Soman, K.P. (2017). Stock Price Prediction Using LSTM, RNN and CNN. *IEEE*.
23. Lai, G., Chang, W.C., Yang, Y., & Liu, H. (2018). Modeling Long- and Short-Term Temporal Patterns. *SIGIR*.
24. Lim, B., Arik, S.Ö., Loeff, N., & Pfister, T. (2021). Temporal Fusion Transformers for Forecasting. *IJF*.
25. Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS*.
26. Che, Z., Purushotham, S., Cho, K., Sontag, D., & Liu, Y. (2018). Recurrent Neural Networks for Multivariate Time Series. *Scientific Reports*.
27. Brownlee, J. (2018). *Deep Learning for Time Series Forecasting*. Machine Learning Mastery.

28. Yahoo Finance (2024). Historical Stock Price Data.
29. Fama, E.F. (1970). Efficient Market Hypothesis. *Journal of Finance*.
30. Tsay, R.S. (2010). *Analysis of Financial Time Series*. Wiley.
31. Bishop, C.M. (2006). *Pattern Recognition and Machine Learning*. Springer.
32. Murphy, K.P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.
33. Kingma, D.P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*.
34. Dietterich, T.G. (2000). *Ensemble Methods in Machine Learning*. Springer.
35. Silver, D., et al. (2016). Mastering the Game of Go with Deep Neural Networks. *Nature*.
36. Zhang, Y., Aggarwal, C., & Qi, G.J. (2017). Stock Price Prediction via Discovering Multi-Frequency Trading Patterns. *KDD*.